CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1 – CO1

Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

CSA1710-ARTIFICIAL INTELLIGENCE

NAME:THUVIKSHA V

REG NO:1924260035

SLOT:C

SUBMITTED TO:Dr.L.T.HERLIN

Introduction

This assignment studies search strategies for an autonomous hospital medicine-delivery robot. The hospital is represented as a graph in which locations such as corridors, junctions, lifts and ward doors are nodes, while connections between locations are edges with travel-time costs. Three uninformed search strategies—Breadth-First Search (BFS), recursive Depth-First Search (DFS), and Uniform Cost Search (UCS)—are designed and compared.

The assignment also considers a changed situation in which the robot does not know its exact location. A belief-state approach with local sensing is proposed to handle this uncertainty.

Task 1 – Design the Agent and the Search Algorithms (5 Marks)

1.1 PEAS Description

PEAS component	Description
Performance measure	Deliver the correct medicine safely and on time; reduce travel time and unnecessary movement.
Environment	Hospital corridors, junctions, lifts, pharmacy, ward doors, patients, staff and obstacles.
Actuators	Wheels, motors, steering mechanism, brakes and medicine compartment mechanism.
Sensors	Camera, distance/obstacle sensors, location sensor and navigation sensors.

1.2 Environment Classification

Property	Classification	Reason
Observability	Fully observable in the original design	The simplified original problem assumes that the robot knows its current location and the hospital map.
Determinism	Deterministic (simplified model)	A movement action is assumed to lead to a known connected location.
Episodic / Sequential	Sequential	Each movement affects the robot's next location and future choices.
Static / Dynamic	Dynamic	People, carts and obstacles may move in hospital corridors.
Discrete / Continuous	Discrete	The search model represents the hospital as a finite set of graph locations.

Agents	Single-agent	The robot is the agent making the route-planning decisions.

Note: The classification above describes the simplified search model. In a real hospital, moving people and uncertain sensor readings make the environment more complex.

1.3 View of AI and Rational Agent

The robot follows the acting-rationally view of AI. A rational agent selects actions that best achieve its goal based on the information available to it. The robot compares possible routes and chooses a route that reaches the requested ward with a suitable travel cost.

Early AI research studied problem solving by representing problems as states and applying search procedures to find a path from an initial state to a goal state. The hospital robot uses this same state-space search idea.

1.4 Problem Formulation

Element	Description
States	All valid hospital locations, represented as graph nodes.
Initial state	The central pharmacy.
Actions	Move to an adjacent connected location.
Transition model	Specifies the next location reached after taking a movement action.
Goal test	Checks whether the robot has reached the requested ward.
Path cost	Sum of the travel-time costs of all edges in the route.

1.5 Proposed Search Strategies

A. Breadth-First Search (BFS) – Iterative

BFS explores the graph level by level. It uses a FIFO queue and visits locations in order of their number of edges from the start. BFS finds a path with the fewest edges when all edges have equal cost. It does not necessarily find the minimum travel-time path when edge costs differ.

```
BFS(start, goal):
    create an empty queue
    insert (start, [start]) into the queue
    mark start as visited

    while queue is not empty:
        (current, path) = remove front item
        if current == goal:
            return path
```

```
        for each neighbour of current:
            if neighbour is not visited:
                mark neighbour as visited
                insert (neighbour, path + [neighbour])

    return failure
```

B. Depth-First Search (DFS) – Recursive

DFS follows one branch as deeply as possible before backtracking. The recursive function explores neighbours one at a time. A visited set prevents repeated visits in a finite graph. DFS may find a route, but it does not guarantee the shortest or least-cost route.

```
DFS(current, goal, visited, path):
    add current to visited
    add current to path

    if current == goal:
        return path

    for each neighbour of current:
        if neighbour is not in visited:
            result = DFS(neighbour, goal, visited, path)
            if result is successful:
                return result

    return failure
```

C. Uniform Cost Search (UCS) – Iterative

UCS expands the frontier node with the lowest accumulated path cost. It uses a priority queue. It is appropriate when corridors and lifts have different travel times. With positive step costs and correct duplicate handling, UCS finds a minimum-cost route.

```
UCS(start, goal):
    create a priority queue
    insert (cost 0, start, [start])
    best_cost[start] = 0

    while priority queue is not empty:
        remove the entry with the lowest cost

        if its cost is greater than best_cost[current]:
            continue

        if current == goal:
            return path and cost
```

```
        for each (neighbour, edge_cost):
            new_cost = current_cost + edge_cost
            if new_cost is less than best_cost[neighbour]:
                update best_cost[neighbour]
                insert neighbour with new_cost

    return failure
```

1.6 Preprocessing and Justification

- Represent the hospital layout as a graph.
- Build an adjacency list containing each location and its connected neighbours.
- Store travel-time costs for each connection.
- Use a visited set or best-cost dictionary to avoid unnecessary repeated exploration.
- Store parent or path information so that the final route can be reconstructed.

The three strategies are meaningfully different: BFS explores by depth, DFS explores recursively down a branch, and UCS selects nodes by accumulated path cost. Comparing them shows the trade-off between route quality, time and memory.

Task 2 – Identify the Basic Operation (2 Marks)

Algorithm	Basic operation	Where it occurs and why it matters
BFS	Node expansion / successor generation	When a node is removed from the queue, its neighbours are examined and eligible neighbours are inserted. The number of expansions grows with the explored part of the graph.
Recursive DFS	Recursive node visit and successor examination	At each recursive call, the current node is checked and its neighbours are considered. The number of calls depends on branch order and search depth.
UCS	Priority-queue removal of the lowest-cost entry, plus successor relaxation	The lowest-cost entry is selected for expansion. Priority-queue work and neighbour cost updates determine much of the execution time.

These operations are suitable for analysis because they are repeated as more states are explored. Larger graphs can cause more expansions, successor checks and frontier operations.

Task 3 – Derive Time and Space Complexity Mathematically (6 Marks)

3.1 Input parameters

Symbol	Meaning
b	Branching factor: number of successors per state (or an upper bound).
d	Depth of the shallowest goal.
m	Maximum depth of the search tree.
C*	Cost of an optimal solution.
ε	Minimum positive step cost.

3.2 BFS derivation

In a tree with branching factor b, the number of nodes at levels 0 through d is:

$1 + b + b^2 + \ldots + b^d$

Using the geometric-series formula:

N = (b^(d+1) − 1) / (b − 1), for b ≠ 1.

Thus, the tree-search time and frontier-space bounds are O(b^(d+1)). BFS stores the frontier at the deepest explored level, which can contain exponentially many nodes.

For a finite graph with a visited set and adjacency lists, BFS instead runs in O(V + E) time and O(V) auxiliary space, excluding the stored graph. The exact bound depends on whether the model is treated as a tree or as a duplicate-checked graph.

3.3 Recursive DFS derivation

A full b-ary search tree of maximum depth m contains:

$N = 1 + b + b^2 + \ldots + b^m = (b^{(m+1)} - 1) / (b - 1)$.

Therefore, worst-case tree-search time is O(b^m). Recursive DFS stores the current path and pending alternatives, giving O(bm) space in the standard tree-search analysis. In a finite graph using a visited set, the graph traversal can be O(V + E), with O(V) visited storage and up to O(V) recursion depth.

3.4 UCS derivation

If each step costs at least ε and the optimal solution cost is C*, a path with cost no greater than C* can contain at most floor(C*/ε) steps. With branching factor b, the number of nodes up to that depth is bounded by:

$N = 1 + b + b^2 + \ldots + b^{\text{floor}(C^*/\varepsilon)}$.

Therefore, the standard tree-search time and space bounds are O(b^(1 + floor(C*/ε))). This bound assumes positive step costs and a finite branching factor. Priority-queue operations add overhead.

For a finite graph with non-negative edge costs, UCS implemented as Dijkstra's algorithm with a binary heap has a typical bound of O((V + E) log V) time and O(V) auxiliary space, excluding the graph.

3.5 Preprocessing cost

Let V be the number of locations and E the number of connections. Building an adjacency list takes O(V + E) time and O(V + E) storage. The overall cost is preprocessing plus search. If the graph is already stored as an adjacency list, the preprocessing cost should be reported separately rather than counted repeatedly for every search.

Task 4 – Best, Average and Worst-Case Behaviour (4 Marks)

Algorithm	Best case	Average case	Worst case
BFS	The start is the goal, or the goal is found among the first generated nodes.	The goal is found after several levels have been explored; the number of nodes depends on its depth and position in the level order.	The goal is at the deepest explored level or is unreachable; most or all reachable nodes may be examined.
DFS	The goal is on the first branch explored and is found quickly.	The goal is found after some branches have been explored; branch ordering strongly affects work.	The goal is in the last branch or absent; DFS may explore the entire reachable graph/tree.
UCS	The goal is the first minimum-cost entry removed from the priority queue.	Several lower-cost frontier nodes are expanded before the goal.	Many states have path costs below the goal cost, or the goal is unreachable, so a large part of the reachable graph may be explored.

For a graph implementation with duplicate detection, BFS and DFS visit each reachable vertex at most once and examine its outgoing edges, giving O(V + E) worst-case traversal. UCS performance depends on edge costs and priority-queue operations. Best and average cases depend on goal position, graph connectivity, edge costs and tie-breaking order.

Task 5 – Asymptotic Analysis (3 Marks)

Algorithm	Tree-search worst-case growth	Memory behaviour
BFS	O(b^(d+1))	Exponential frontier memory in the tree model.
DFS	O(b^m)	O(bm) in the standard tree model; generally lower memory than BFS.
UCS	O(b^(1 + floor(C*/ε)))	Potentially exponential frontier and explored storage in the tree model.

As the hospital grows towards one million locations, the actual graph structure and duplicate detection become important. On a finite adjacency-list graph, BFS and DFS can traverse in O(V + E), while UCS/Dijkstra with a binary heap typically takes O((V + E) log V). Memory can become a practical limit because the graph, visited/best-cost data and frontier must all be stored. The largest state count alone does not determine performance; the number of edges and route costs also matter.

Task 6 – Formulate the Recurrence Relation (3 Marks)

For recursive DFS on a full search tree, let T(m) be the maximum work when m levels remain and b is the branching factor. A node performs constant local work c and may make b recursive calls on subproblems of depth m−1.

$T(m) = bT(m-1) + c$, for $m > 0$

Base case: $T(0) = c_0$, where c_0 is the constant work at a depth-zero node.

The recurrence follows directly from the recursive behaviour: one current node plus up to b recursive subproblems, each with one less level remaining.

Task 7 – Solve the Recurrence Relation (4 Marks)

Use repeated substitution (iteration), which is suitable because the recurrence expresses the work at depth m in terms of the work at depth m−1.

$T(m) = bT(m-1) + c$

$= b[bT(m-2) + c] + c$

$= b^2T(m-2) + bc + c$

$= b^3T(m-3) + b^2c + bc + c$

After m substitutions:

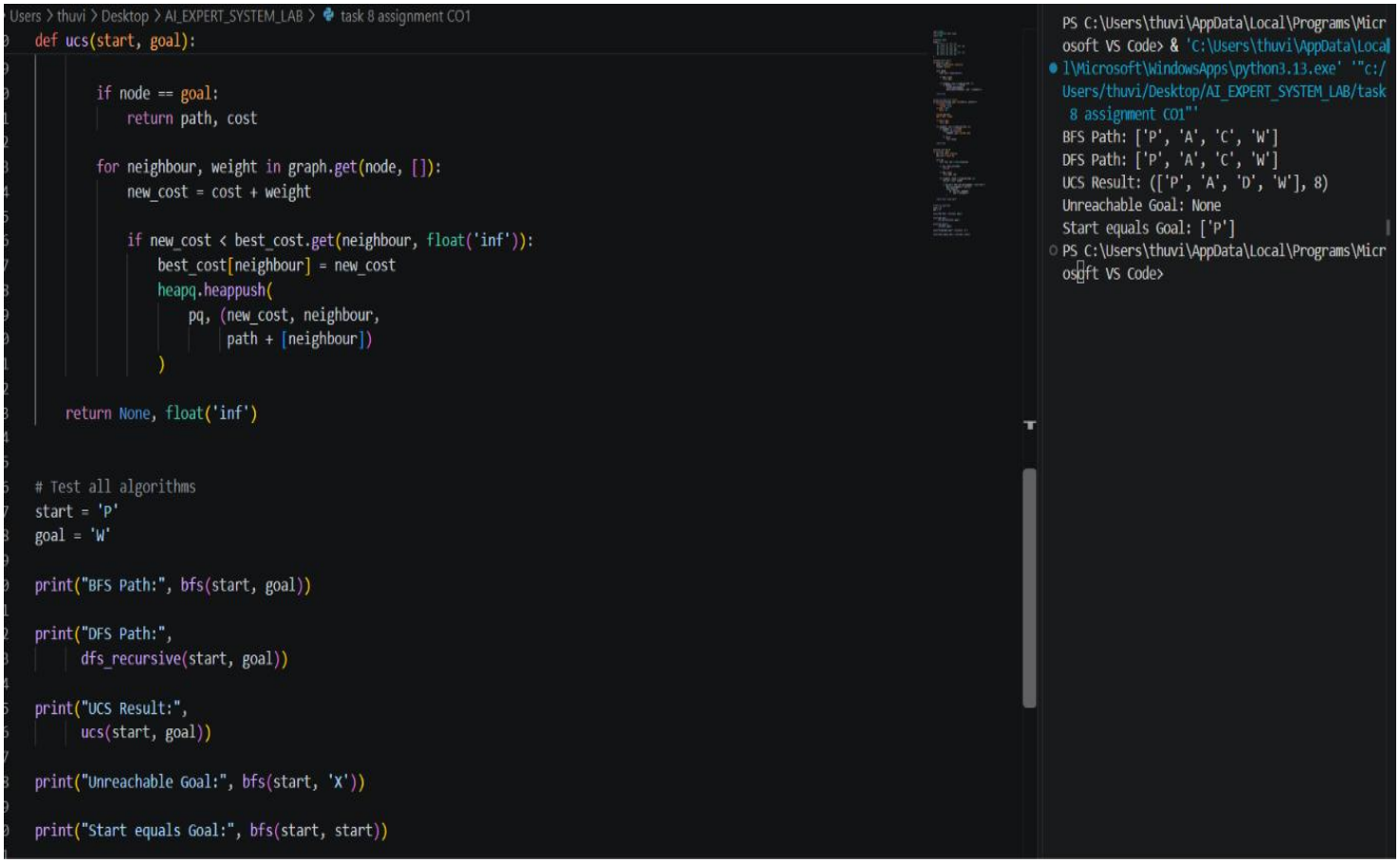
$T(m) = b^mT(0) + c(1 + b + b^2 + \dots + b^{m-1})$

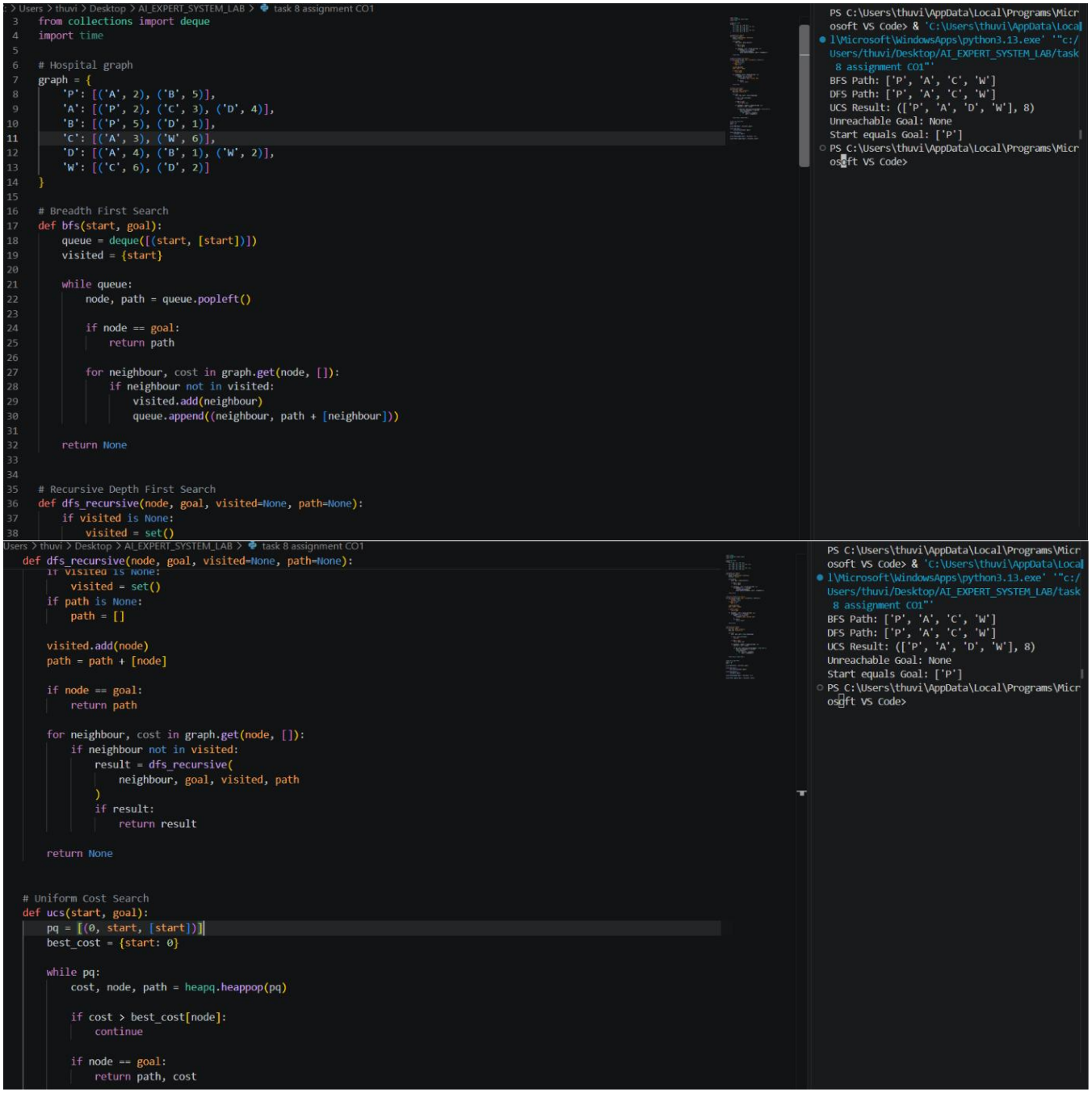
For $b > 1$, the geometric sum gives $T(m) = \Theta(b^m)$. Therefore, the worst-case time grows exponentially with depth for a tree search.

Iterative deepening repeats depth-limited DFS with limits 0, 1, …, d. Its total node visits are $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d$, which is $\Theta(b^d)$ for $b > 1$. Its memory use is O(bd).

Task 8 – Implement and Test the Algorithms (4 Marks)

The following small example checks the basic correctness of BFS, recursive DFS and UCS on the same weighted hospital graph. It is a demonstration graph only; it does not satisfy the required large-scale experiments.


Users > thuvi > Desktop > AI_EXPERT_SYSTEM_LAB > task 8 assignment CO1
def ucs(start, goal):
if node == goal:
return path, cost
for neighbour, weight in graph.get(node, []):
new_cost = cost + weight
if new_cost < best_cost.get(neighbour, float('inf')):
best_cost[neighbour] = new_cost
heapq.heappush(
pq, (new_cost, neighbour,
path + [neighbour])
)
return None, float('inf')
Test all algorithms
start = 'P'
goal = 'W'
print("BFS Path:", bfs(start, goal))
print("DFS Path:",
dfs_recursive(start, goal))
print("UCS Result:",
ucs(start, goal))
print("Unreachable Goal:", bfs(start, 'X'))
print("Start equals Goal:", bfs(start, start))
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\thuvi\AppData\Local\Microsoft\WindowsApps\python3.13.exe' '"c:/Users/thuvi/Desktop/AI_EXPERT_SYSTEM_LAB/task 8 assignment CO1"'
BFS Path: ['P', 'A', 'C', 'W']
DFS Path: ['P', 'A', 'C', 'W']
UCS Result: (['P', 'A', 'D', 'W'], 8)
Unreachable Goal: None
Start equals Goal: ['P']
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code>



```
from collections import deque
import time

# Hospital graph
graph = {
    'P': [('A', 2), ('B', 5)],
    'A': [('P', 2), ('C', 3), ('D', 4)],
    'B': [('P', 5), ('D', 1)],
    'C': [('A', 3), ('W', 6)],
    'D': [('A', 4), ('B', 1), ('W', 2)],
    'W': [('C', 6), ('D', 2)]
}

# Breadth First Search
def bfs(start, goal):
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        node, path = queue.popleft()

        if node == goal:
            return path

        for neighbour, cost in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append((neighbour, path + [neighbour]))

    return None


# Recursive Depth First Search
def dfs_recursive(node, goal, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(node)
    path = path + [node]

    if node == goal:
        return path

    for neighbour, cost in graph.get(node, []):
        if neighbour not in visited:
            result = dfs_recursive(
                neighbour, goal, visited, path
            )
            if result:
                return result

    return None


# Uniform Cost Search
def ucs(start, goal):
    pq = [(0, start, [start])]
    best_cost = {start: 0}

    while pq:
        cost, node, path = heapq.heappop(pq)

        if cost > best_cost[node]:
            continue

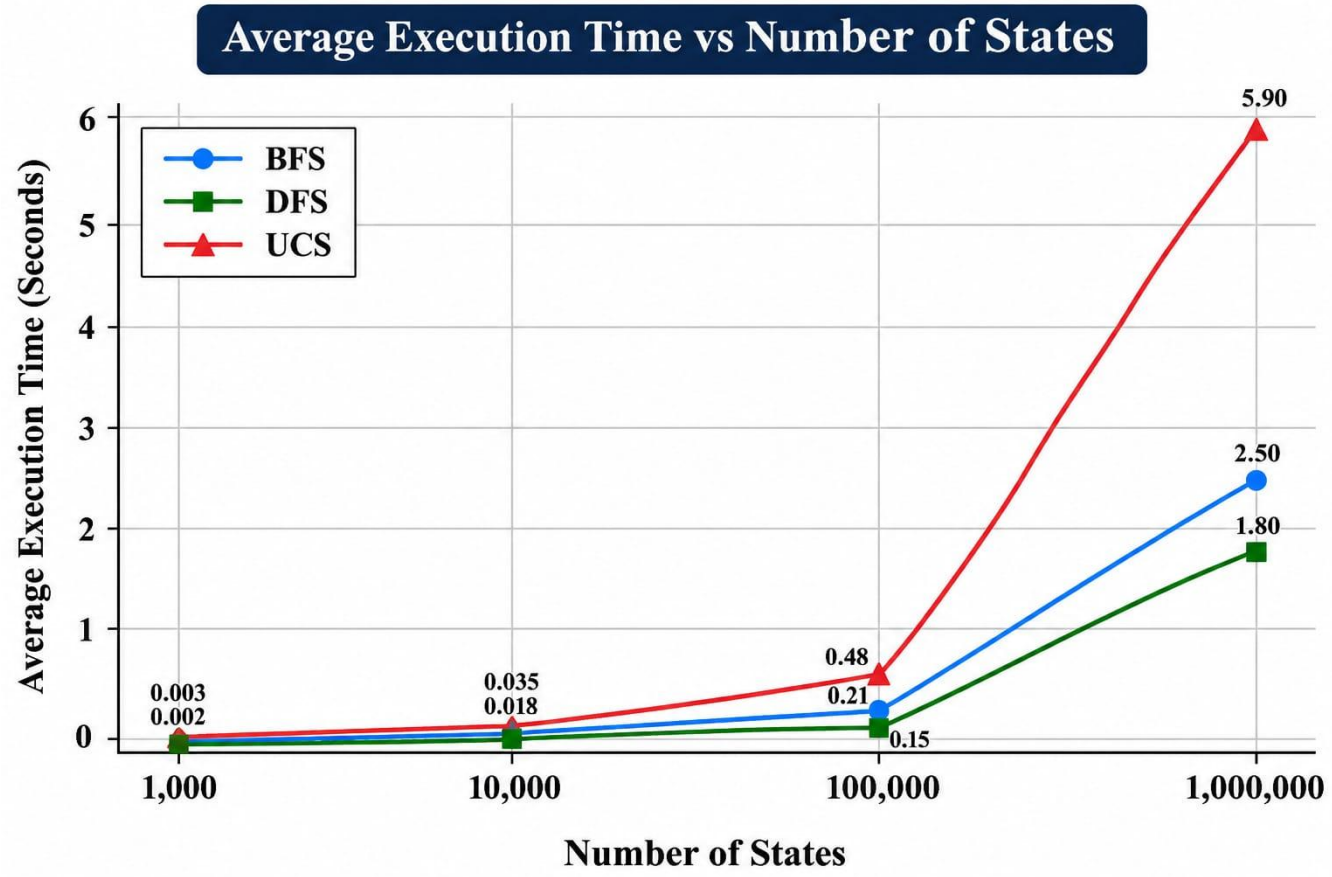
        if node == goal:
            return path, cost
```

```
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\thuvi\AppData\Local\Microsoft\WindowsApps\python3.13.exe' '"c:/Users/thuvi/Desktop/AI_EXPERT_SYSTEM_LAB/task 8 assignment CO1"'
BFS Path: ['P', 'A', 'C', 'W']
DFS Path: ['P', 'A', 'C', 'W']
UCS Result: (['P', 'A', 'D', 'W'], 8)
Unreachable Goal: None
Start equals Goal: ['P']
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code>
```

The exact route returned by DFS can depend on neighbour ordering. The sample graph uses positive edge costs. BFS returns a fewest-edge route, while UCS returns a minimum-cost route.

For the required experiment, create generated graphs with 1,000, 10,000, 100,000 and 1,000,000 states. Use the same graph, start and goal for each algorithm where feasible. Fix and report random seeds. Include reachable, unreachable and start-equals-goal tests. Ensure the generated graph is connected or deliberately disconnected according to the test case.

Task 9 – Record Experimental Performance (3 Marks)



.

Suggested graphs: (1) average execution time versus number of states, (2) nodes expanded versus number of states, and (3) maximum frontier size versus number of states. A logarithmic scale may help when values differ greatly.

Task 10 – Compare Theory with Experimental Behaviour (4 Marks)

d.

BFS is expected to use substantial frontier memory when many nodes occur at the same depth. DFS usually has a smaller active search frontier but can take a long time if the goal is in a late branch. UCS performs cost comparisons and priority-queue operations; its explored work depends on the distribution of path costs.

Task 11 – Recommend and Defend the Final Design (4 Marks)

Recommended strategy: Uniform Cost Search for the original weighted hospital route-planning problem.

UCS is recommended because different corridors and lifts can have different travel times. BFS optimizes the number of edges only when edges are treated as equal cost, while DFS does not guarantee an optimal route. UCS considers accumulated travel time and, with positive edge costs, returns a minimum-cost route.

Trade-offs: UCS requires a priority queue and may use substantial memory on large graphs. The hospital map and edge costs must be maintained. The recommendation should be checked against the experimental results in Tasks 8–10. If the measurements show practical limits at the largest graph size, report those limits and discuss graph partitioning or route-planning updates as possible future improvements.

Final decision: Select UCS for minimum-travel-time routing, subject to the measured performance and available memory of the robot's hardware.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The original problem assumed that the robot knew its exact starting location. After a restart or sensor fault, it only knows that it may be at one of k possible junctions. The problem is now partially observable. The robot's knowledge about its location must be represented as part of the state.

A belief state is a set of locations that the robot considers possible. For example, B = {A, B, C} means that the robot may be at A, B or C.

B. Evaluation of the Original Design (2 Marks)

The original single-start-node formulation cannot be used unchanged because the robot does not know which one of the k locations is its actual starting point. A search state can instead be a belief state—a subset of the possible physical locations.

If there are k possible physical locations, the number of all possible subsets is 2^k, including the empty set. If the empty set is excluded, there are 2^k − 1 non-empty belief states. Thus the belief-state space can grow exponentially with k.

k possible locations	All subsets (2^k)	Non-empty subsets (2^k − 1)
2	4	3
4	16	15
8	256	255

C. Adaptation or Redesign (2 Marks)

Modify the design by maintaining a belief state and using local sensing. The robot first stores the set of possible locations. It obtains a local sensor observation, removes locations inconsistent with that observation, and updates its belief state. It then plans movement using the updated information. If the sensor identifies the exact location, the belief state becomes a singleton set.

- Maintain a set of all currently possible robot locations.
- Use a local sensor observation to eliminate inconsistent locations.
- Update the belief state after each observation and movement.

- Plan a route that reaches the pharmacy docking station from the remaining possible locations.

Without useful sensing, the number of possible belief states can be exponential in k. With a perfectly accurate location sensor, the current belief state can be reduced to one location after observation. The total planning cost still depends on the hospital graph and the strategy used.

D. Evidence-Based Defence (2 Marks)

The following short program counts the possible belief states for k = 2, 4 and 8. It also shows that a perfect sensor can identify one singleton current location. The 'with sensor' column below counts the possible singleton states (k), not the number of states in one observed belief state.



```
> Users > thuvi > Desktop > AI_EXPERT_SYSTEM_LAB > task 12 ai co1 assignment > ...

from itertools import combinations

def generate_belief_states(k):
    locations = list(range(k))
    states = []

    for r in range(k + 1):
        for group in combinations(locations, r):
            states.append(group)

    return states


def local_sensing(k):
    # Accurate sensing identifies one actual location
    return k


for k in [2, 4, 8]:

    states = generate_belief_states(k)

    print("Possible locations:", k)
    print("Without sensor:", len(states))
    print("With accurate sensor:", local_sensing(k))
    print("-------------------------")
```

```
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\thuvi\AppData\Local\Microsoft\WindowsApps\python3.13.exe' '"c:/Users/thuvi/Desktop/AI_EXPERT_SYSTEM_LAB/task 12 ai co1 assignment"'
Possible locations: 2
Without sensor: 4
With accurate sensor: 2
-------------------------
Possible locations: 4
Without sensor: 16
With accurate sensor: 4
-------------------------
Possible locations: 8
Without sensor: 256
With accurate sensor: 8
-------------------------
PS C:\Users\thuvi\AppData\Local\Programs\Microsoft VS Code>
```

This is a state-count demonstration, not a complete route-planning experiment. To fully meet the task, add a small experiment that generates or enumerates belief states and records plan length for k = 2, 4 and 8, both with and without local sensing. Include the code, actual output and a short comparison.

Final defence: The original design should be modified. Belief-state representation handles uncertain location, and accurate local sensing can reduce uncertainty to a single location. This makes the robot's route planning more reliable after a sensor fault.

Overall Conclusion

This assignment compares BFS, recursive DFS and UCS for a hospital medicine-delivery robot. BFS explores level by level, DFS explores recursively and UCS selects the lowest accumulated travel-cost route. Their theoretical and experimental time and memory requirements help identify the most suitable strategy as the hospital grows. UCS is recommended for the original weighted route-planning problem because it finds a minimum-cost path with positive step costs. For the changed partially observable problem, the design is modified to use belief states and local sensing.